

Microservices, Message brokers and Apache kafka

1. The concept of Microservices

Most traditional applications are monolithic. They are developed, deployed and scaled as a single unit. Each team working on the application should always coordinate with each other. They need to make sure they are not affecting each others work. All the teams follow the same tech stack. On every update that's being made we need to redeploy the entire applications. Considering these things there are multiple challenges we face while following this architecture. Few of them which can be mentioned are,

- ✓ Applications become too large and complex to handle.
- ✓ Scaling can be done only to the entire app rather than a specific service.
- ✓ High infrastructural costs.
- ✓ Difficulties arrive when different services needs different dependency versions.
- ✓ Bug in any module can potentially bring down an entire application.

So an alternative to overcome these challenges would be to follow the concept of a microservice architecture. In a microservice architecture, we break applications into smaller pieces or in other words smaller services. It is split mainly based on business functionality rather than its technical functionality. Every service has one specific job. Every service is self-contained and independently developed, deployed and scaled.

The teams assigned for each service don't have to be worried about other services and can work using any tech stack they desire. Each service can communicate with each other only when required. Some of the ways by which this can be done is,

- ✓ Using API endpoints- Where each service sends messages in the form of HTTP requests and gets an HTTP response. This is a form of synchronous communication where the request and response is immediate from each side
- ✓ Using message brokers- Where each service sends messages to a broker which in turn forwards it to the required services. This is a form of asynchronous communication.
- ✓ Using a service mesh- The service mesh uses local sidecar proxies rather than plugging directly into the service instances.

Some of the challenges faced in a while using a microservice architecture are,

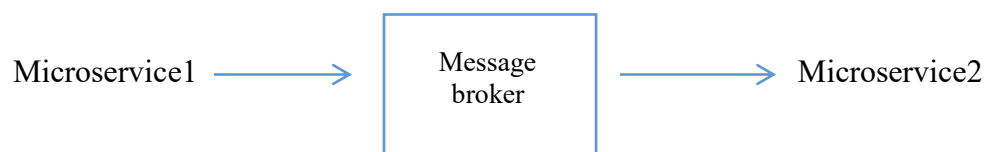
- ✓ Configuring communications within the services might get complicated when many microservices are deployed.
- ✓ Difficult to monitor with multiple instances of each service distributed across servers.
- ✓ If one instance fails it could affect multiple other instances within the organization, since it is highly distributed. It also becomes difficult to track the root of the problem.

Some popular tools are used to contain and tackle these problems. For example,

1. Contanerization- Docker
2. Container orchestraisation- Kubernetes
3. Pipeline automation- GitLab
4. Asynchronous messaging- Kafka
5. Performance monitoring- Prometheus
6. Logging and auditing- DataDog

Communication between microservices plays a vital role in the functioning of any full grade application. So in this documentation we focus on how microservices communicate with each using message brokers and Apache Kafka, a distributed event store and stream-processing platform.

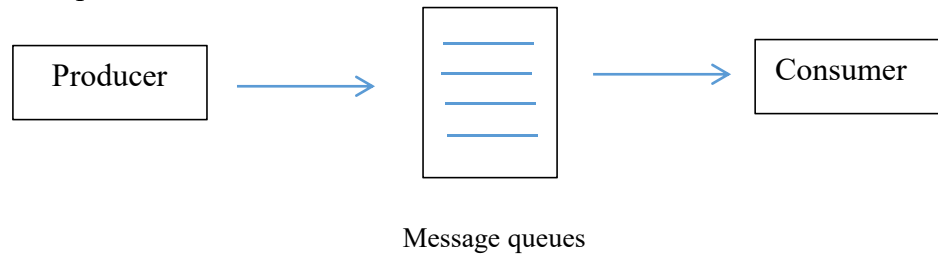
2. Message brokers



Message brokers work as intermediaries that helps in communication between microservices or applicatons. All message brokers involve message queues. So its important to understand message queues first.

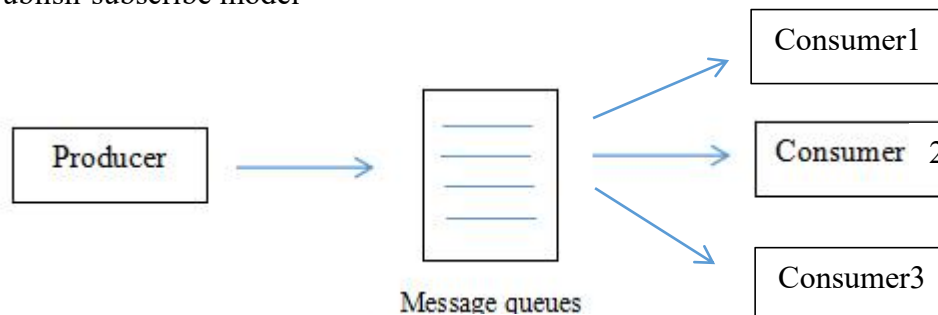
Message queues ensure asynchronous communication. They follow first in first out method. There can be two types of communication between MS when message queues are involved within a message broker.

1. Point to point model



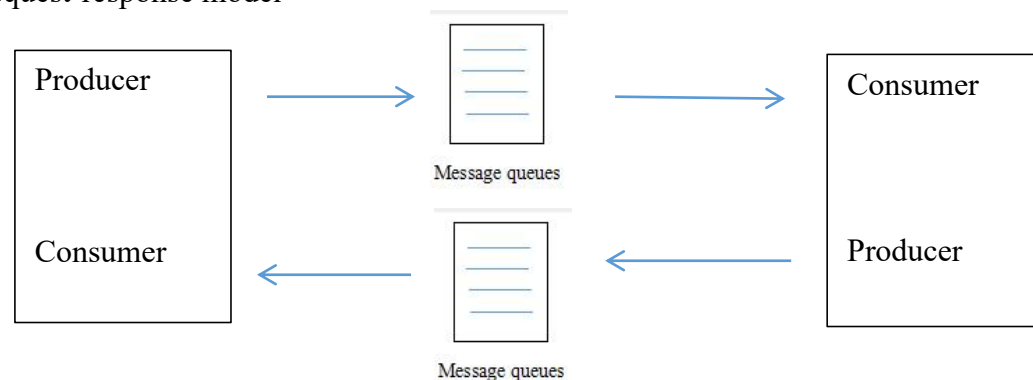
In point to point model there is one producer and one consumer. Each message sent to the message queue is forwarded only to one consumer. The consumer can access the messages whenever he is available in the order it was sent.

2. Publish-subscribe model



In publish-subscribe model multiple consumers subscribe to a particular topic and the producer publishes the messages to those topics. Only the subscribed consumers can access the messages.

3. Request-response model

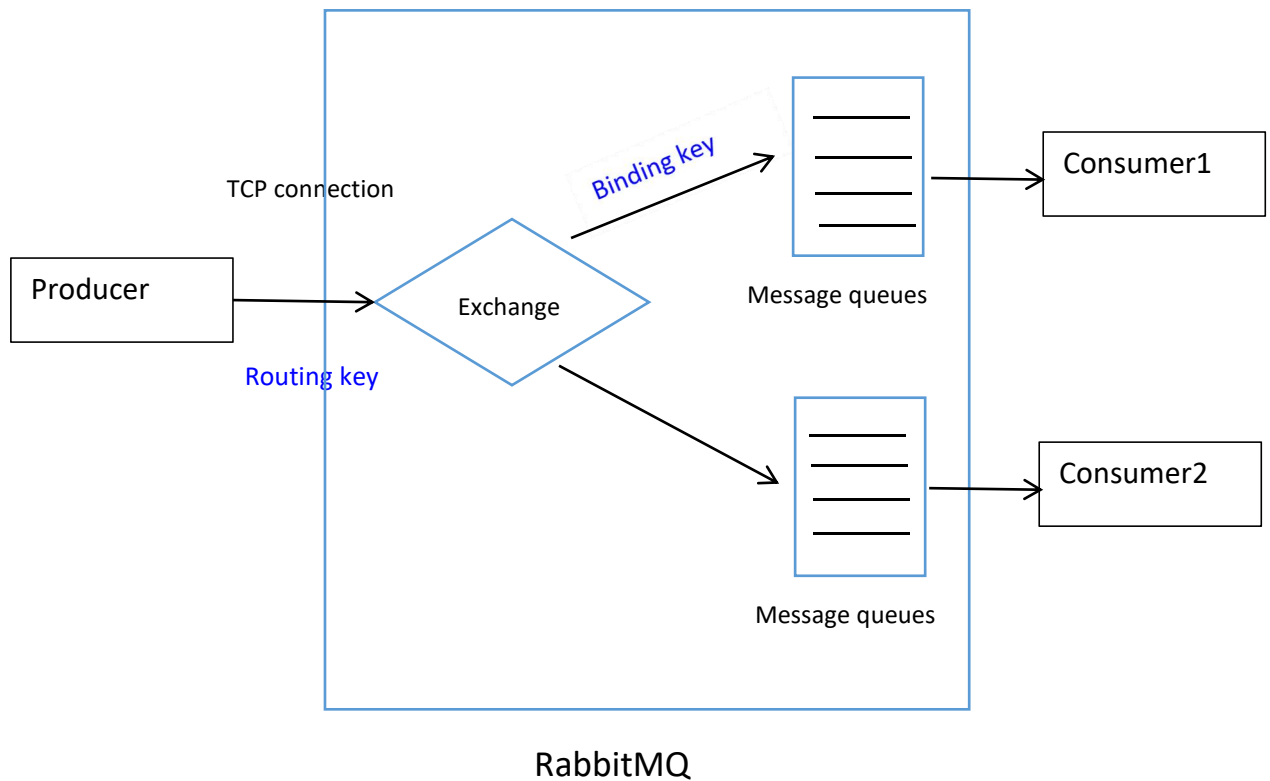


In this model, Producer sends a message to the consumer and also possible to receive a response from the consumer. In the later case the then consumer becomes the producer and the then producer becomes the consumer. So there goes on a request and response of messages from both the sides.

Now we will be moving on to exploring two different message brokers, RabbitMQ and ActiveMQ

3. RabbitMQ

RabbitMQ follows the Advanced message queuing protocol. The working of RabbitMQ can be depicted through the following diagram.



From the diagram we can take away the following points,

- ✓ The producer first sends the message to the exchange present in the broker and then the exchange connects to the message queue. Then it is sent to the consumer.
- ✓ There are several ways the communication takes places with different types of exchanges
 1. Fanout exchange- Producer sends out message to exchange and exchange duplicates the message to every single queue it knows about.
 2. Direct exchange- Producer produces a message with a routing key. If it matches the binding key, only then the messages are passed to the queues.
 3. Topic exchange- This exchange happens when there is even a partial match between the binding key and the routing key.
 4. Header exchange- Routing key is completely ignored in this case and the message is moved through system according to header.
 5. Default exchange- In this case the routing key is tied to the name of the queue itself.
- ✓ Once the messages reaches the queue, the messages are sent to the concerned consumers accordingly.

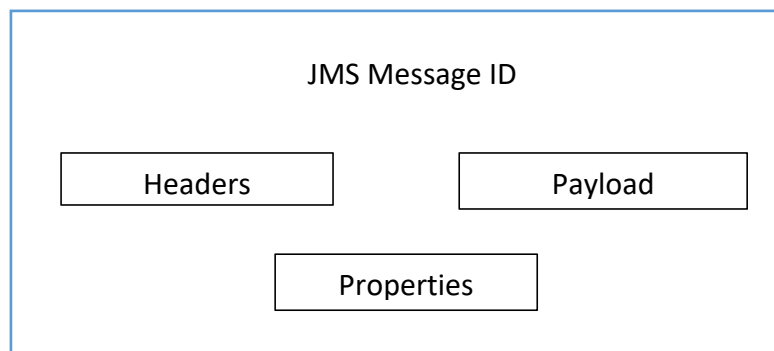
Some of the benefits associated with RabbitMQ is that,

- ✓ It is cloud friendly.

- ✓ The cluster is highly fault tolerant, high throughput, Highly available.
- ✓ The developer and the application has a lot of control over how the message moves rather than broker administrator.
- ✓ It supports cross language communication.
- ✓ Message stays in the message queue until consumer acknowledges that message is received.
- ✓ It also supports different message models.

4. ActiveMQ

- ✓ It is a message broker that implements java messaging service(JMS) which allows applications to send data back and forth between one another in an easy standard way.
- ✓ In this applications are considered as endpoints. There are two types of endpoints. One, the inbound endpoints that we call the consumers and two, the outbound endpoints that we call the producers.
- ✓ It involves three things- messages, queues, topics
- ✓ A JMS message has the following architecture. It consists of
 1. JMS message ID- To identify the message
 2. Header
 3. Payload- Could be the content of the message
 4. Properties- that is associated to the message.



- ✓ The messages that come in are arranged in two patterns- queues and topics.
 1. Queues follow the FIFO(First in first out) pipeline. It has a load balancing work flow. The producer pushes the messages to the queue and consumer collects one message at a time.
 2. Topics follow a publish subscribe workflow. They have subscription based message broadcast channels. Recipients subscribed to the topic receives broadcast of the message sent by the producing application. So the producer is called the publisher and the consumer is called the subscriber.

5. Apache Kafka

Before understanding kafka it is important to understand event streaming. Event streaming is capturing real time data in the form of events- manipulating, processing and reacting to them and routing them to different destinations.

Apache kafka performs the following functions

- ✓ To publish and subscribe streams of events- continuous imports and exports
- ✓ To store streams of events- durability and reliability
- ✓ To process streams of events

Kafka has better throughput, built in partitioning replication and inherent fault tolerance. It has publish subscribe messaging system and robust queue that can handle high volumes of data. It is suitable for both online and offline message consumption. It also performs operational monitoring of data, collects logs from multiple services and stream processing.

Some of the advantages of using the Apache kafka are that it gives,

- ✓ High throughput
- ✓ High scalability
- ✓ Low latency
- ✓ Permanent storage
- ✓ High availability

Generally used for,

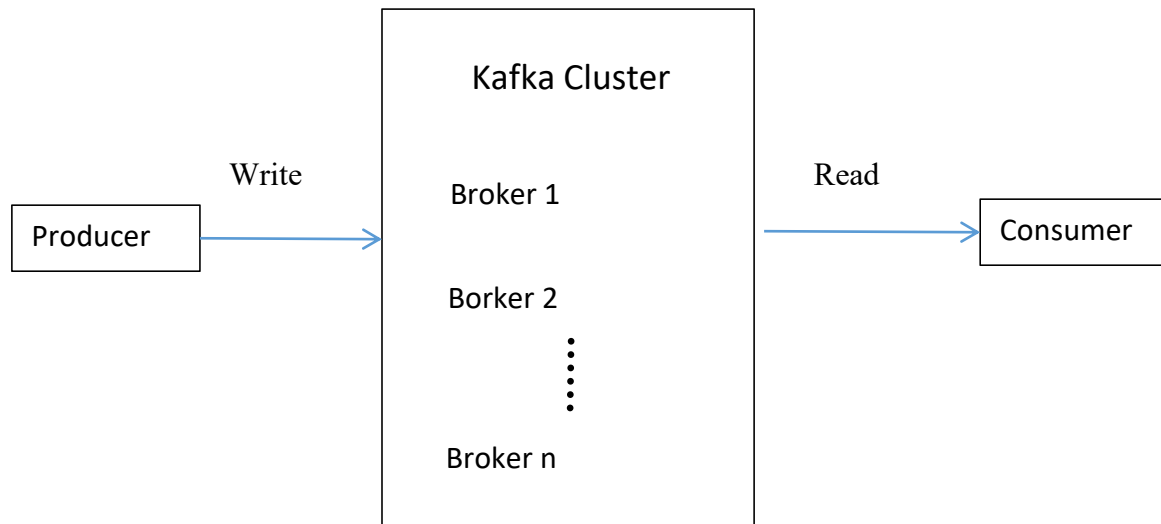
- ✓ Data pipelines- Ingesting and streaming of data
- ✓ Stream processing- Perform different operations and other data transformations.
- ✓ Streaming analytics
- ✓ Streaming ETL- ETL stands for extract transform and load that is the process of combining data from multiple sources into large central repository called data warehouse.
- ✓ Event driven microservices.

Apache kafka contains several message brokers within it. A kafka broker mainly deals with events that are nothing but the messages or records sent by the application. An event basically contains the following items,

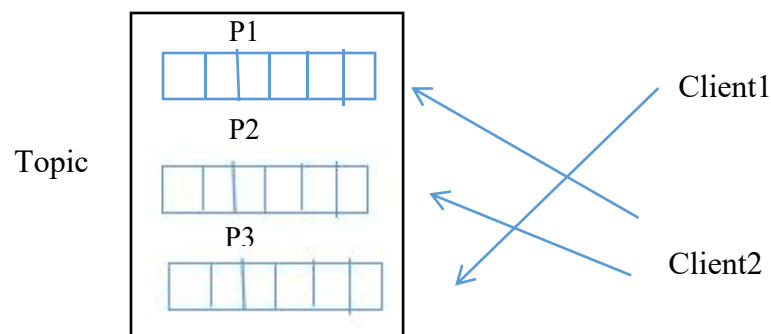
- ✓ Key
- ✓ Value
- ✓ Timestamp
- ✓ Optional meta data headers

Now let's see what are things that happen in kafka.

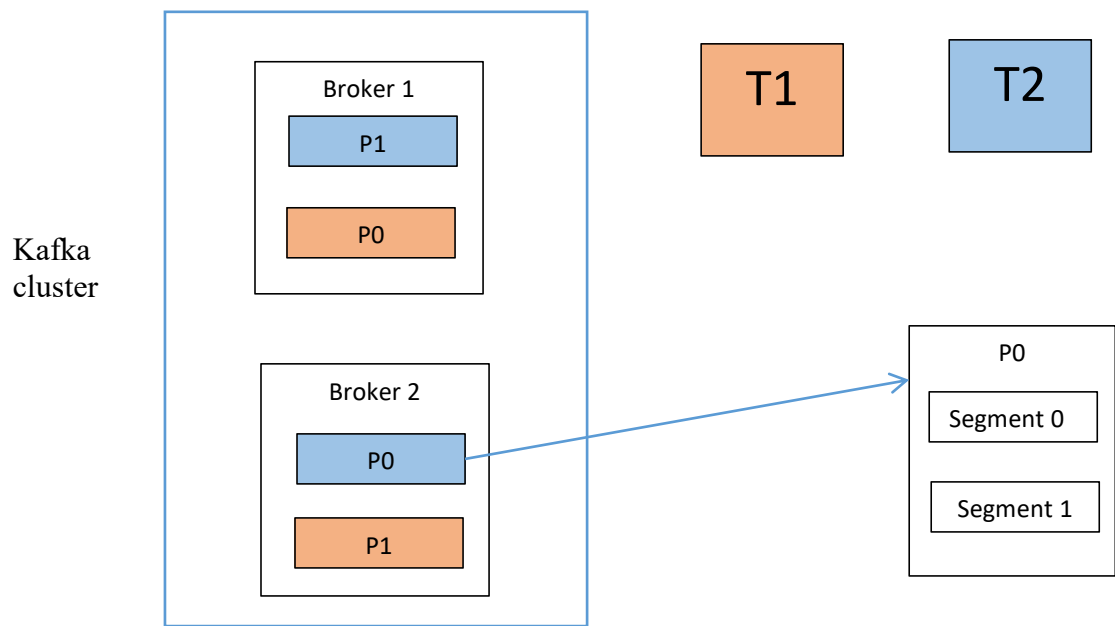
- ✓ The basic function would be as follows- Producer writes to the kafka cluster which has several brokers with it. These messages or data from the producer are then sent to consumers who have subscribed to it. This kafka cluster is also connected to zookeeper. Zookeeper keeps tracks of failure and helps in recovery. Also it does cluster management. This can be represented as a diagram as below.



- ✓ Events from producer are stored in topics.
- ✓ Topics can have many producers that write events to it. Similarly it can contain many consumers that subscribe to these events.
- ✓ Stores data even after consumption. We can decide how long it can be stored.
- ✓ Topics are partitioned or spread over number of buckets located on different kafka broker.
- ✓ Events with same event key is written into same partitions.
- ✓ Topics can be replicated and can be present across regions or data centers.



- ✓ The above diagram shows that two different clients are subscribed to the same topic, which has multiple partitions. Two different clients can access messages from the same partition based on their requirements.
- ✓ The structure of Kafka can be depicted as below
- ✓ As it can be seen kafka contains a kafka cluster. It contains several brokers within it. Partitions within a topic are stored across different brokers. Each partition can be further divided as segments within it. As seen in the diagram topics and partitions that are related are denoted with the same colour.
- ✓ Kafka also supports different APIs like Admin API, Producer API, Consumer API, Kafka streams API, Kafka connect API.



6. RabbitMQ vs ActiveMQ vs Kafka

Properties	Kafka	RabbitMQ	ActiveMQ
Performance and Scalability	High throughput in terms of huge volume data	Comparatively lower than kafka when data volume is considered	Comparatively lower than kafka when data volume is considered
Message Ordering	Ensures message ordering within a partition but not across partitions within a topic.	Guarantee message ordering within a single queue or topic	Guarantee message ordering within a single queue or topic
Message Priority	Does not have built-in message priority support.	Support message prioritization, allowing messages with higher priority to be processed before those with lower priority.	Support message prioritization, allowing messages with higher priority to be processed before those with lower priority.
Message Model	Utilizes a distributed log-based model	Queue-based message model following the Advanced Message Queuing Protocol (AMQP)	Built on the Java 5. Message Service (JMS) standard and also uses a queue-based message model.
Replication	Features built-in partition replication	Supports replication through Mirrored Queues	Master-Slave replication mechanism

Stream Processing	Native stream processing capabilities through Kafka Streams	Also offers stream processing	Relies on third-party libraries for stream processing.
-------------------	---	-------------------------------	--

7. Kafka implementation on windows with simple message communication between two topics

Steps Involved

1. Java should be installed .
2. Download kafka from official website.
3. Extract the tgz folder into desired location.(in my case d drive)
4. Open the kafka/config/zookeeper.properties file and add the path **d:\kafka\zookeeper-data** in **dataDir**
5. Then open the kafka/config/server.properties file and add the path **d:\kafka\kafka-logs** in **logs.dirs**
6. Open 5 different command prompts
 - i. Start zookeeper in kafka directory- **.\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties**
 - ii. Run kafka server in kafka directory- **.\bin\windows\kafka-server-start.bat .\config\server.properties**
 - iii. Create topic test1 in kafka\bin\windows directory- **kafka-topics.bat --create --topic test1 --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1**
 - iv. Create producer for topic test1 in kafka\bin\windows directory- **kafka-console-producer.bat --broker-list localhost:9092 --topic test1**

Input message >Hello

>World

- v. Create consumer for topic test1 in kafka\bin\windows directory- **kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic test1 --from-beginning**

Messages received from producer will be displayed >Hello

>World